

Lab 4: 4-bit Carry-Lookahead Adder

CMPE 125 — Section 02

Name: Yixu Liu

Student ID: 017406857

Instructor: Jahan Ghofraniha

Date: October 6, 2026

1. Design and Calculations

The purpose of this lab was to design a 4-bit carry-lookahead adder (CLA) using structural Verilog, simulate it in Vivado, and test it on a Basys 3 FPGA board. The circuit adds two 4-bit unsigned inputs, A and B , and one carry input, C_i . It produces a 4-bit sum S , carry output C_o , group propagate PG , and group generate GG . The complete unsigned result is the 5-bit value $\{C_o, S\}$.

1.1 Module Structure

The design uses the three modules required by the lab manual.

Module	Function
GPFullAdder	Uses XOR and AND gates to calculate one bit's propagate, generate, and sum.
CLALogic	Uses AND and OR gates to calculate the internal carries, C_o , PG , and GG .
CLA4	Connects four GPFullAdder instances to one CLALogic instance.

1.2 Logic Equations

For bit i , the propagate, generate, and sum signals are

$$P_i = A_i \oplus B_i, \quad G_i = A_i B_i, \quad S_i = P_i \oplus C_i.$$

Here, $\oplus$ means XOR, multiplication means AND, and $+$ in the carry equations means OR. A bit generates a carry when both inputs are 1, and propagates an incoming carry when exactly one input is 1. Let $C_0 = C_i$. Expanding $C_{i+1} = G_i + P_i C_i$ gives

$$C_1 = G_0 + P_0 C_0,$$

$$C_2 = G_1 + P_1 G_0 + P_1 P_0 C_0,$$

$$C_3 = G_2 + P_2 G_1 + P_2 P_1 G_0 + P_2 P_1 P_0 C_0,$$

$$C_4 = G_3 + P_3 G_2 + P_3 P_2 G_1 + P_3 P_2 P_1 G_0 + P_3 P_2 P_1 P_0 C_0.$$

The group signals and carry output are

$$PG = P_3 P_2 P_1 P_0,$$

$$GG = G_3 + P_3 G_2 + P_3 P_2 G_1 + P_3 P_2 P_1 G_0,$$

$$C_o = C_4 = GG + PG C_0.$$

$PG = 1$ means the block passes an incoming carry through all four bits. $GG = 1$ means the block produces a carry even when $C_i = 0$. In the code, $C[0]$, $C[1]$, and $C[2]$ represent C_1 , C_2 , and C_3 . These feed the full adders for bits 1, 2, and 3; bit 0 receives C_i directly. The expanded equations avoid calculating each carry from the preceding carry output.

1.3 Test Calculations

Test 1: $5 + 3$, with $C_i = 0$. The inputs are $A = 0101_2$ and $B = 0011_2$. Therefore, $P = 0110_2$ and $G = 0001_2$. The carry equations give $(C_1, C_2, C_3, C_4) = (1, 1, 1, 0)$, so

$$S = 0110_2 \oplus 1110_2 = 1000_2, \quad \{C_o, S\} = 0\ 1000_2 = 8_{10}.$$

The expected group outputs are $PG = 0$ and $GG = 0$.

Test 2: $15 + 15$, with $C_i = 0$. The inputs are $A = 1111_2$ and $B = 1111_2$. Therefore, $P = 0000_2$ and $G = 1111_2$. All four carry outputs are 1, so

$$S = 0000_2 \oplus 1110_2 = 1110_2, \quad \{C_o, S\} = 1\ 1110_2 = 30_{10}.$$

The expected group outputs are $PG = 0$ and $GG = 1$. The carry output is needed because 30 cannot fit in four unsigned bits.

2. Simulation Waveforms

The testbench applies $5 + 3$ from 0 to 10 ns and $15 + 15$ from 10 to 20 ns. C_i remains 0 in both tests. The Vivado waveform viewer displays the 4-bit buses in hexadecimal, so **f** means 15 and **e** means 14. The unused lower area of each screenshot is cropped for readability.

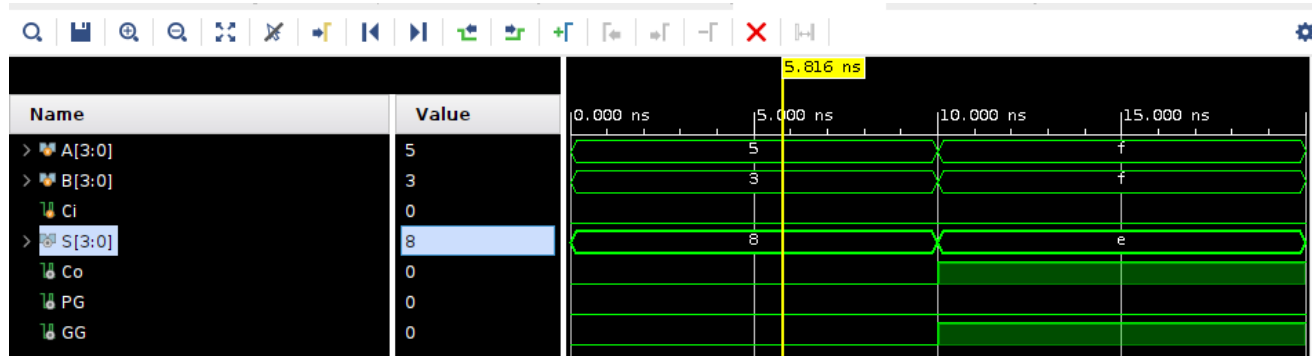


Figure 1: First test selected in Vivado: $A = 5$, $B = 3$, $S = 8$, and $C_o = 0$.

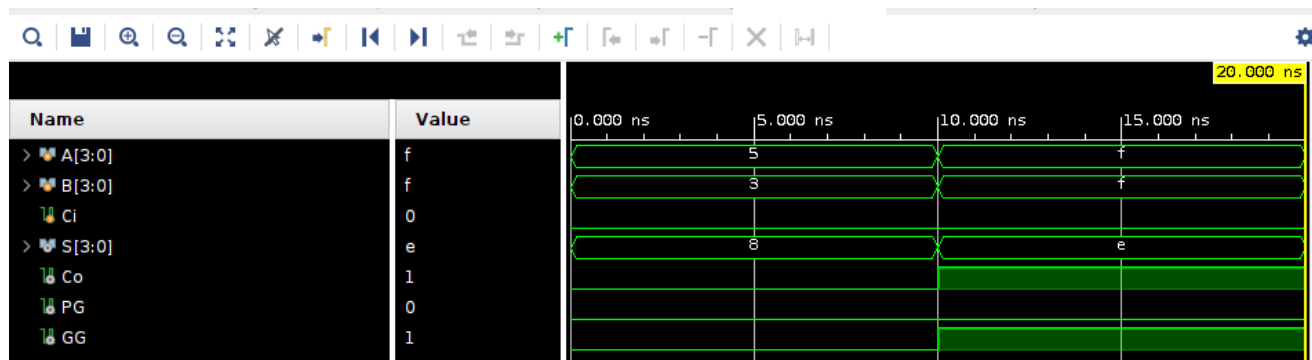


Figure 2: Second test selected in Vivado: $A = \text{f}$, $B = \text{f}$, $S = \text{e}$, and $C_o = 1$, representing a total of 30.

Table 1: Expected and observed simulation results. Buses are shown in binary.

A	B	C_i	Expected $\{C_o, S\}$	Observed $\{C_o, S\}$	PG	GG	Result
0101	0011	0	0 1000	0 1000	0	0	Pass
1111	1111	0	1 1110	1 1110	0	1	Pass

3. Verification Summary

The simulation results matched the calculated outputs for both addition tests. For $5 + 3$, the adder produced $S = 1000_2$ with $C_o = 0$, giving a total of 8. For $15 + 15$, it produced $S = 1110_2$ with $C_o = 1$, giving a total of 30. The group outputs also matched the equations: $PG = 0$ in both tests, while GG changed from 0 to 1 in the second test. These results verify the sum and carry behavior for the two required test cases. The waveforms are functional simulations; they do not measure the circuit's physical propagation delay.

4. FPGA Board Tests

The Basys 3 photographs document three hardware test cases: $10 + 5$, $10 + 5 + 1$, and $15 + 15$. Table 2 lists their expected outputs. The first two cases show why the carry input matters: adding 1 to 15 changes the 4-bit sum from 1111_2 to 0000_2 and sets the carry output. The sum is read together with C_o to obtain the complete result.

Table 2: Expected outputs for the photographed board tests.

Test	C_i	$S[3:0]$	C_o	PG	GG	Total
$10 + 5$	0	1111	0	1	0	15
$10 + 5 + 1$	1	0000	1	1	0	16
$15 + 15$	0	1110	1	0	1	30

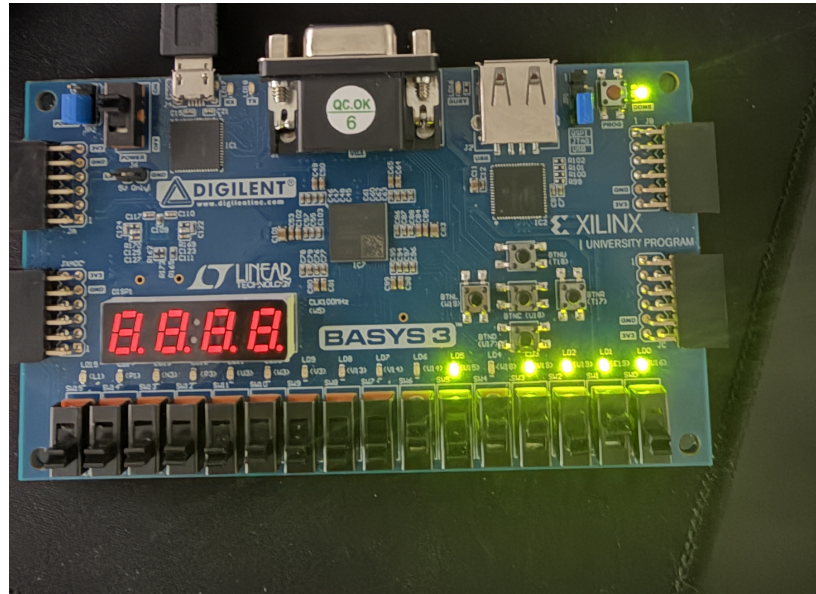


Figure 3: Basys 3 test: $10 + 5 = 15$.

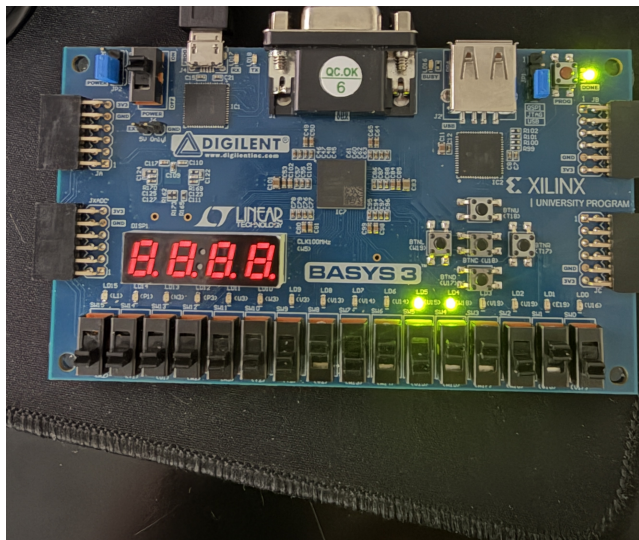


Figure 4: Basys 3 test: $10 + 5 + 1 = 16$.

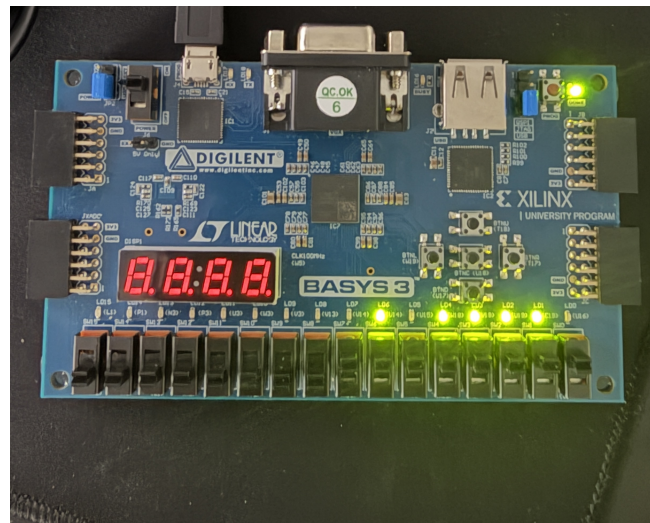


Figure 5: Basys 3 test: $15 + 15 = 30$.

Sources: CMPE 125, *Lab 4: 4-bit CLA Adder* manual; CMPE 125 *Lab Report Format*; lab Verilog files, Vivado waveform screenshots, and Basys 3 photographs.

5. Verilog Code

The complete design and testbench are included below. Empty tool-generated headers were omitted and spacing was shortened for readability. The second test comment was corrected; the circuit logic and test inputs are unchanged.

5.1 GPFullAdder

```
`timescale 1ns/1ps

module GPFullAdder (
    input  Ai,
    input  Bi,
    input  Cin,
    output G,
    output P,
    output S
);

    xor (P, Ai, Bi);
    and (G, Ai, Bi);
    xor (S, P, Cin);

endmodule
```

5.2 CLA4

```
`timescale 1ns/1ps

module CLA4 (
    input  [3:0] A,
    input  [3:0] B,
    input      Ci,
    output [3:0] S,
    output      Co,
    output      PG,
    output      GG
);

    wire [3:0] P;
    wire [3:0] G;
    wire [2:0] C;

    // Four 1-bit GP Full Adders
    GPFullAdder FA0 (A[0], B[0], Ci, G[0], P[0], S[0]);

    GPFullAdder FA1 (A[1], B[1], C[0], G[1], P[1], S[1]);

    GPFullAdder FA2 (A[2], B[2], C[1], G[2], P[2], S[2]);

    GPFullAdder FA3 (A[3], B[3], C[2], G[3], P[3], S[3]);

    // Carry Lookahead Logic
    CLALogic CLA (G, P, Ci, C, Co, PG, GG);

endmodule
```

5.3 CLALogic

```

`timescale 1ns/1ps

module CLALogic (
    input  [3:0] G,
    input  [3:0] P,
    input      Ci,
    output [2:0] C,
    output      Co,
    output      PG,
    output      GG
);

    // C1 = G0 + P0*Ci
    wire c1_t1;

    and (c1_t1, P[0], Ci);
    or  (C[0], G[0], c1_t1);

    // C2 = G1 + P1*G0 + P1*P0*Ci
    wire c2_t1;
    wire c2_t2;

    and (c2_t1, P[1], G[0]);
    and (c2_t2, P[1], P[0], Ci);
    or  (C[1], G[1], c2_t1, c2_t2);

    // C3 = G2 + P2*G1 + P2*P1*G0 + P2*P1*P0*Ci
    wire c3_t1;
    wire c3_t2;
    wire c3_t3;

    and (c3_t1, P[2], G[1]);
    and (c3_t2, P[2], P[1], G[0]);
    and (c3_t3, P[2], P[1], P[0], Ci);
    or  (C[2], G[2], c3_t1, c3_t2, c3_t3);

    // PG = P3*P2*P1*P0
    and (PG, P[3], P[2], P[1], P[0]);

    // GG = G3 + P3*G2 + P3*P2*G1 + P3*P2*P1*G0
    wire gg_t1;
    wire gg_t2;
    wire gg_t3;

    and (gg_t1, P[3], G[2]);
    and (gg_t2, P[3], P[2], G[1]);
    and (gg_t3, P[3], P[2], P[1], G[0]);
    or  (GG, G[3], gg_t1, gg_t2, gg_t3);

    // Co = GG + PG*Ci
    wire co_t1;

    and (co_t1, PG, Ci);
    or  (Co, GG, co_t1);

endmodule

```

5.4 CLA4_tb

```

`timescale 1ns/1ps

module CLA4_tb;

    reg [3:0] A;
    reg [3:0] B;
    reg      Ci;

    wire [3:0] S;
    wire      Co;
    wire      PG;
    wire      GG;

    // Instantiate the CLA4 circuit
    CLA4 DUT (
        .A(A),
        .B(B),
        .Ci(Ci),
        .S(S),
        .Co(Co),
        .PG(PG),
        .GG(GG)
    );

    initial begin

        // Test 1
        // 5 + 3 = 8
        // 0101 + 0011 = 1000
        A = 4'b0101;
        B = 4'b0011;
        Ci = 1'b0;

        #10;

        $display("Test 1:");
        $display("A=%b B=%b Ci=%b -> S=%b Co=%b",
            A, B, Ci, S, Co);

        // Test 2
        // 15 + 15 = 30
        // 1111 + 1111 = 1 1110
        A = 4'b1111;
        B = 4'b1111;
        Ci = 1'b0;

        #10;

        $display("Test 2:");
        $display("A=%b B=%b Ci=%b -> S=%b Co=%b",
            A, B, Ci, S, Co);

        $stop;

    end

endmodule

```